

DSA Interview Roadmap

Phase 1: Choose One Programming Language

- * C++
- * Java
- * Python

Master:

- * Variables
 - * Loops
 - * Functions
 - * Recursion
 - * Time & Space Complexity
 - * Input/Output
-

Phase 2: Learn Libraries

C++

- * vector
- * array
- * string
- * pair
- * stack
- * queue
- * deque
- * priority_queue
- * set
- * unordered_set
- * map
- * unordered_map
- * multiset
- * algorithm
- * bitset

Know

- * insertion complexity
- * deletion complexity
- * searching complexity
- * iteration
- * common STL functions

Java Collections

- * ArrayList
- * LinkedList
- * Stack
- * Queue
- * PriorityQueue
- * HashMap
- * TreeMap
- * HashSet
- * TreeSet
- * Deque

Python Collections

`collections`

- * `deque`
- * `Counter`

- * defaultdict
- * OrderedDict

heapq

bisect

itertools

math

queue

Know what each library is used for.

Phase 3: Learn Every Data Structure

For every data structure learn

- * Definition
- * Internal Working
- * Time Complexity
- * Space Complexity

- * Implementation
- * Interview Uses

Then solve basic implementation questions.

Example

Arrays

Learn

- * insertion
- * deletion
- * traversal
- * searching
- * reversing
- * rotation
- * prefix sum

Practice

- * Find largest element
- * Second largest
- * Remove duplicates

- * Left rotate
 - * Move zeros
 - * Prefix sum
-

Strings

Learn

- * immutable vs mutable
- * substring
- * concatenation
- * frequency counting

Practice

- * palindrome
 - * reverse
 - * anagram
 - * longest common prefix
-

Linked List

Learn

- * Singly
- * Doubly
- * Circular

Practice

- * insert at beginning
- * insert at end
- * delete beginning
- * delete end
- * reverse
- * middle node
- * detect cycle
- * merge lists

Stack

Practice

- * push
 - * pop
 - * peek
 - * balanced parentheses
 - * infix to postfix
 - * postfix evaluation
 - * next greater element
-

Queue

Practice

- * enqueue
 - * dequeue
 - * circular queue
 - * deque
 - * priority queue
-

HashMap

Practice

- * insert
 - * delete
 - * search
 - * frequency counting
 - * pair sum
-

Heap

Practice

- * build heap
 - * heapify
 - * priority queue
 - * kth largest
 - * kth smallest
-

Tree

Practice

- * inorder
- * preorder
- * postorder
- * level order
- * height
- * diameter
- * balanced tree
- * LCA

BST

Practice

- * insert
- * delete
- * search
- * floor
- * ceil

Graph

Practice

- * adjacency list
- * adjacency matrix
- * BFS
- * DFS
- * cycle detection
- * topological sort
- * shortest path

Trie

Practice

- * insert
- * search
- * startsWith
- * delete

Union Find

Practice

- * union
- * find
- * path compression
- * rank

Phase 4: Pattern-wise Practice

For every pattern

- * Learn intuition
- * Learn clues
- * Learn template
- * Learn complexity
- * Solve Easy
- * Solve Medium

* Solve Hard

* Revise

25 Major Patterns

1. Two Pointers
2. Sliding Window
3. Prefix Sum
4. Merge Intervals
5. Binary Search
6. Sorting
7. Fast & Slow Pointer
8. Backtracking
9. Divide and Conquer
10. Linked List
11. Stack
12. Monotonic Stack
13. Expression Evaluation
14. String
15. Hashing
16. Trees
17. Path Sum

- 18. Heap
- 19. Top K
- 20. Merge K Lists
- 21. Dynamic Programming
- 22. Greedy
- 23. BFS & DFS
- 24. Graph Algorithms
- 25. Design Problems

DSA Pattern Sheet:

https://docs.google.com/document/d/1Es-TFTTmHrEZ3ZY-W5H_NgUy41tegf4BNP9SajzUrME/